

Requirements Document — E-Commerce Backend Database

Course: CS4092 — Database Design and Development, Summer 2026 **Author:** Caleb Yost **Project:** Individual — Backend database for a small e-commerce platform

1. Purpose

This project designs and implements a relational database backing a small e-commerce platform. The system tracks customers, the products they can buy, staff members who manage inventory, the credit cards customers pay with, and the purchases customers make. It supports two user roles — **Staff** and **Customer** — each with distinct responsibilities and permitted actions.

2. Stakeholders / User Roles

Role	Description
Staff	Employees responsible for maintaining the product catalog and inventory levels.
Customer	End users who browse products, manage payment methods, and make purchases.

3. Functional Requirements

3.1 Staff

- FR-1: A staff member can add a new product to the catalog (name, price, starting stock quantity).
- FR-2: A staff member can edit an existing product's name, price, or stock quantity.
- FR-3: A staff member can view current inventory levels for all products.
- FR-4: The system records which staff member added each product.

3.2 Customer

- FR-5: A customer can browse the list of available products (name, price, stock available).
- FR-6: A customer can register one or more credit cards to their account.
- FR-7: A customer can purchase a product using one of their registered credit cards.

- FR-8: A purchase must reduce the product's stock quantity by the quantity purchased.
- FR-9: A customer cannot purchase more units of a product than are currently in stock.

3.3 Reporting / Queries

- FR-10: The system can report which customers purchased which products, restricted by product price (e.g., products over \$100).
- FR-11: The system can report total revenue or units sold per product.
- FR-12: The system can report each customer's total spend across all purchases.

4. Non-Functional Requirements

- NFR-1: Every entity has a unique, system-generated primary key.
- NFR-2: Referential integrity is enforced between related tables (foreign keys).
- NFR-3: The schema supports at least 4 relations per the individual project requirement (this design uses 5: Customer, Staff, Product, CreditCard, Purchase).
- NFR-4: The database is implemented in a relational DBMS (SQLite for this project — see README for rationale) and is accessible from a command-line Python application.

5. Use Cases

UC-1: Staff adds a product Actor: Staff. Staff selects "Add Product," enters name/price/quantity. System creates a new Product row linked to the staff member's staff_id and confirms success.

UC-2: Staff updates inventory Actor: Staff. Staff selects a product and enters a new stock quantity or price. System updates the Product row.

UC-3: Customer registers a credit card Actor: Customer. Customer selects "Add Credit Card" and enters card details. System creates a CreditCard row linked to the customer's customer_id.

UC-4: Customer purchases a product Actor: Customer. Customer browses products, selects one, chooses a registered card, and enters a quantity. System validates stock is sufficient, creates a Purchase row, and decrements the product's stock.

UC-5: Staff or analyst runs a sales report Actor: Staff. Staff runs a query to see, e.g., which customers bought which products priced over \$100, or total revenue per product.

6. Out of Scope

- Payment processing / card validation with a real payment gateway.

- Returns, refunds, shipping/delivery tracking.
- Multi-seller / marketplace features.
- Authentication and session management beyond role selection at the CLI.